

From Chain to DAG — The Vertex Model

Why Hathor arranges blocks and transactions in one directed acyclic graph instead of a chain of blocks — and the two kinds of edge that hold it together.

SUBJECT hathor-core · Part I · Track B (domain concepts)

CHAPTER 08 · Foundations · Concepts

BRANCH study/hathor-core-studies

EDITION 2026 · v1

Graph · DAG · Topological order · Vertex · Block vs Transaction · Parents vs Inputs · Tips · Confirmation · Merged structure

Table of Contents

Chapter 8 — From Chain to DAG: The Vertex Model	3
8.1 Just enough graph theory	3
8.2 The limit of a single chain	4
8.3 The DAG of vertices	5
8.4 The crux: parents versus inputs	5
8.5 What blocks are still for, tips, and confirmation	6
8.6 Bridge — the vertex model in code	7
Recap	8

Chapter 8 — From Chain to DAG: The Vertex Model

What you'll learn in this chapter

- Enough **graph theory** to read the rest of the book: nodes, edges, directed, acyclic, and why "acyclic" is non-negotiable for a ledger.
- The limitation of a single **chain of blocks**, and what Hathor gains by letting transactions link directly to each other.
- The **vertex**: Hathor's umbrella term for "a block or a transaction," and the real class hierarchy that implements it.
- **The crux of this whole book**: the two distinct kinds of edge a vertex has — **parents** (DAG/confirmation topology) and **inputs** (which coins it spends) — and why confusing them makes the codebase unreadable.
- What **blocks** are still for in a DAG (ordering and minting), what **tips** are, and how a new vertex **confirms** older ones.
- A **bridge** to the code that defines and walks the graph.

This is the chapter that gives the project its shape, and the one a newcomer most needs to get right. Chapter 6 described the classic design — transactions bundled into blocks, blocks in a single chain. Hathor keeps blocks but dissolves the bundling: ordinary transactions become first-class citizens of the ledger graph, linking directly to one another. The result is a **DAG**, and once you hold its structure — especially the two-kinds-of-edge distinction in §8.4 — the transaction code, the consensus, and the sync protocol all become legible.

We build the graph vocabulary first (§8.1), because the rest depends on it, then motivate the DAG (§8.2–8.3), then the edges (§8.4), then Hathor's classes (§8.5).

8.1 Just enough graph theory

A **graph**¹ is one of the simplest structures in computer science: a set of **nodes**² (also called vertices) connected by **edges** (links). That's all. A social network is a graph (people = nodes, friendships = edges); a road map is a graph (intersections = nodes, roads = edges).

Two refinements turn a plain graph into the kind Hathor uses:

Directed. In a **directed graph**³, each edge has a direction — it points from one node to another, like a one-way street. "A → B" is not the same as "B → A." Hathor's edges are directed: a transaction points back to the earlier ones it builds on.

Acyclic. A **cycle** is a path along the edges that returns to where it started (A → B → C → A). A graph with no cycles is **acyclic**⁴. Put the two together and you get a **DAG** — a Directed Acyclic Graph: edges have direction, and you can never follow them in a loop.

A DAG (edges point one way; no path loops back)



Following arrows from A you reach B, C, D — never back to A.

Why must a ledger be acyclic? Because the edges encode "this came after that." A transaction points to earlier transactions it builds upon. If a cycle existed — X after Y after X — you'd have a contradiction in time: X both precedes and follows itself. Acyclicity is what lets the graph express a consistent history. It also guarantees a **topological order**⁵: you can always lay the nodes out in a line such that every edge points backward, i.e. every vertex comes after everything it depends on. That ordering is what makes it possible to process the graph (verify a vertex only after its dependencies, replay history deterministically) — and it is why a DAG can still serve as a ledger even without a single built-in chain.

8.2 The limit of a single chain

Recall the classic structure (Chapter 6): transactions are collected into blocks, and blocks form one chain, each pointing to its predecessor. This is itself a trivial DAG — a straight line, the simplest acyclic directed graph. It works, but the single-file shape has consequences:

- **Transactions don't exist on the ledger until a block includes them.** Your transaction waits in a holding area (the mempool) until a miner packs it into a block. Until then it is, ledger-wise, in limbo.
- **Throughput is gated by blocks.** History grows one block at a time, at the block interval, so the rate at which transactions become part of the ledger is capped by block production.
- **Everything funnels through miners.** Only miners extend the ledger; ordinary users produce transactions but cannot themselves grow the structure.

Hathor's design question was: what if a transaction could attach to the ledger directly — pointing at earlier transactions — without waiting to be packed into a block? If transactions can reference each other, the single line opens up into a branching, merging graph, and several of the above constraints loosen. That is the move from chain to DAG.

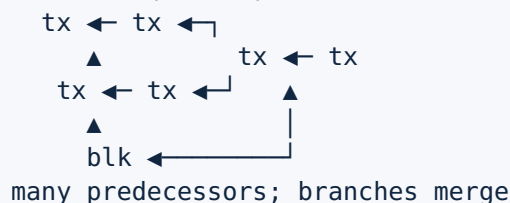
8.3 The DAG of vertices

In Hathor, the ledger is one DAG whose nodes are both blocks and transactions. When a new transaction is created, it points back at a small number of earlier vertices already in the graph — and by doing so, it attaches itself to the ledger immediately, no block required. Because many transactions can attach in parallel, and each can point at more than one predecessor, the structure branches and re-merges into a mesh rather than a line.

A CHAIN (classic)
blk ◀ blk ◀ blk ◀ blk

one predecessor each
(a single line)

A DAG (Hathor) – schematic



Because blocks and transactions live in the same graph and share most of their structure, the codebase needs one word for "a node of this graph, whichever kind it is." That word is **vertex**⁶ — the graph-theory term from §8.1, used throughout `hathor-core`. A vertex is either a block or a transaction; when the code doesn't care which, it says "vertex."

Two questions immediately arise, and the next two sections answer them. First: what exactly are the edges — what does it mean for one vertex to "point at" another? (It turns out there are **two** different kinds, and conflating them is the classic beginner error.) Second: if transactions no longer need blocks, what are blocks still for?

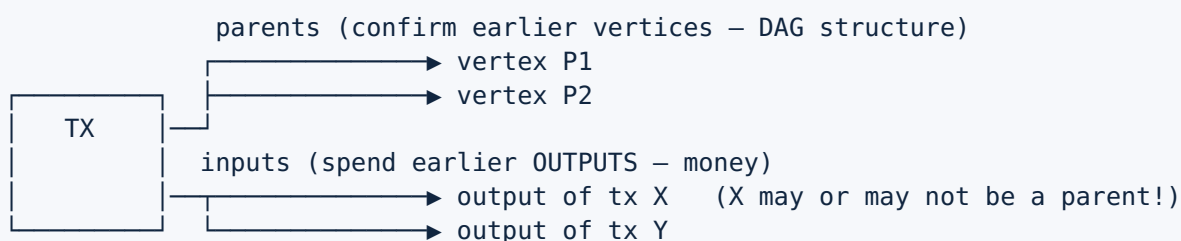
8.4 The crux: parents versus inputs

This is the single most consequential distinction in the book. A Hathor vertex has two separate kinds of outgoing edge, which serve completely different purposes. Beginners conflate them and then find the code incomprehensible. Keep them apart and most of the system clicks.

Parents — the DAG/confirmation topology. Every vertex names a small list of **parents**⁷: earlier vertices it attaches to and thereby confirms. This is the structural edge — the analogue of the classic block's `prev` hash pointer (Chapter 6), but generalized: instead of one predecessor, a vertex has a few. Parents are what make the graph a graph; they encode "this vertex comes after those, and vouches for them." In the code, this is the `parents` field, a list of vertex hashes (`base_transaction.py:179`).

Inputs — the spending edges. Separately, a transaction names **inputs**: pointers to specific earlier outputs it spends (the UTXO mechanism of Chapter 7). This edge is about money — which coins are being consumed — not about graph structure. In the code, this is the `inputs` field, a list of `TxInput` objects, each holding a `(tx_id, index)` pair (`base_transaction.py:936`).

ONE TRANSACTION, TWO KINDS OF EDGE



PARENTS answer "what does this confirm / come after?" (topology)
INPUTS answer "what coins does this spend?" (value)

The two edge sets are independent. A transaction you spend from (an input) need not be one of your parents, and a parent need not be one you spend from. The code treats them as distinct sets — `get_all_dependencies()` deliberately unions both (`base_transaction.py`), precisely because to process a vertex you need everything it depends on either way, but the two roles never merge. Internalize this picture; everything in Chapters 25, 31, and 32 leans on it.

One immediate consequence worth stating: a **double-spend** (Chapter 6–7) is a collision in the input edges — two transactions whose inputs point at the same output — and it is entirely possible for both of those transactions to sit happily in the DAG via their parent edges. The graph structure does not prevent conflicts; it contains them, and consensus (Chapter 10, 32) resolves which conflicting transaction wins by **voiding** the other. The DAG holds both; voiding picks one.

8.5 What blocks are still for, tips, and confirmation

If transactions attach themselves directly, **why keep blocks at all?** Two jobs only blocks do:

1. Minting. New coins must enter the supply somewhere. As in Chapter 6, blocks carry the **reward** — outputs with no inputs — so block production is how HTR is created. Transactions move existing money; blocks make new money.

2. A backbone of ordering. A pure transaction-DAG has no single built-in timeline — many transactions attach in parallel, so "which came first" can be ambiguous. Blocks solve this: they form a chain through the DAG (each block has a block-parent), and that chain acts as a spine that pins down a global ordering and a notion of accumulated work (Chapter 9). Transactions are confirmed by blocks — a transaction becomes increasingly settled as blocks pile up that reach back to it through parent edges.

This is why a **block** in Hathor has a richer parent structure than a transaction. The grounding bears this out: a block has **three parents** — one block parent (continuing the block backbone, always `parents[0]` , `block.py`) plus two transaction parents (attaching the block to the transaction-DAG it confirms). A transaction typically has **two parents**, both other transactions/blocks. The code even exposes this asymmetry: `get_tx_parents()` returns `self.parents[1:]` for a block (dropping the block parent) but all parents for a transaction (`base_transaction.py`).

Two more terms you'll meet constantly:

Tips. The **tips**⁸ of the DAG are the vertices that no other vertex has confirmed yet — the current frontier of the graph. When you create a new transaction, you choose your parents from the tips, thereby confirming them and extending the frontier. Selecting good tips to confirm is part of the protocol (and part of what the mempool tracks — Ch 28).

Confirmation. A vertex is **confirmed**⁹ by every later vertex that can reach it by following parent edges backward. The more vertices (and especially blocks) that reach back to yours, the more work stands behind it, and — exactly as with depth in Chapter 6 — the harder it is to reverse. Confirmation in a DAG is the generalization of "blocks built on top" from a chain.

8.6 Bridge — the vertex model in code

The concepts above are implemented in `hathor/transaction/` , toured fully in Chapter 25. The forward-pointers:

Bridge — the DAG and vertices in the codebase (full treatment in the chapters named):

- **The real base class.** The umbrella type is `GenericVertex[StaticMetadataT]`, an abstract generic base (`base_transaction.py:148`). For historical reasons the names `Vertex` and `BaseTransaction` are aliases of it (`base_transaction.py:933`, `BaseTransaction: TypeAlias = Vertex`) — so when you see `BaseTransaction` in older code, read "any vertex." The generic type parameter ties each vertex kind to its own static metadata (Ch 25). This uses the generics of Chapter 5 and the inheritance/ABC of Chapter 1 — **Chapter 25**.
- **The subclasses.** `Block` (`block.py:45`) and `Transaction` (`transaction.py:55`) are the two main kinds; further specializations exist — `MergeMinedBlock` (`merge_mined_block.py:31`) and `PoaBlock` extend `Block`; `TokenCreationTransaction` and `OnChainBlueprint` (`on_chain_blueprint.py:155`) extend `Transaction`. This is the §1.5 inheritance hierarchy at the heart of the ledger — **Chapters 25, 32, 39**.
- **Parents vs. inputs, in fields.** `parents` (`base_transaction.py:179`) is the topology edge; `inputs` (a list of `TxInput`, `base_transaction.py:936`) is the spending edge. The §8.4 distinction is right there in the class — **Chapter 25**.
- **Walking the graph.** Traversing the DAG (ancestors, descendants, in topological order) is done by dedicated walk utilities in `transaction/storage/` — the §8.1 topological order, made operational — **Chapter 27**.
- **Confirmation, weight, and ordering.** How blocks impose ordering and how confirmation accrues work is the weight story — **Chapter 9** — and how conflicts in the input edges are resolved is voiding — **Chapters 10 & 32**.
- **Genesis as the graph's root.** The DAG bottoms out at the genesis vertices (the first block and two genesis transactions), identified by hash (`transaction/genesis.py`), recalled from **Chapter 22** — every parent chain eventually reaches them.

Recap

Concept	Meaning	In Hathor
Graph / node / edge	nodes joined by links	the ledger itself
Directed	edges point one way	a vertex points back at earlier ones
Acyclic (DAG)	no path loops back	enables a consistent history + topological order

Concept	Meaning	In Hathor
Vertex	a node of the ledger graph	<code>GenericVertex</code> (alias <code>BaseTransaction</code>)
Parents	confirmation/topology edge	<code>parents</code> field; block has 3, tx has ~2
Inputs	spending edge (UTXO)	<code>inputs</code> field, <code>TxInput(tx_id, index)</code>
Block (still needed)	minting + ordering backbone	reward outputs; block-parent chain
Tips	unconfirmed frontier	chosen as parents by new vertices
Confirmation	reachable by later vertices	settles a vertex as work accrues

Hathor's ledger is a single directed acyclic graph in which blocks and transactions are both vertices, transactions attach themselves directly by naming **parents**, and value flows through a separate set of **input** edges — two distinct kinds of link that must never be confused. Blocks survive the move from chain to DAG because they alone mint coins and provide an ordering backbone; transactions gain the ability to join the ledger without waiting on a block. The graph holds conflicting transactions side by side and defers to consensus to pick winners. You now have the ledger's structure (this chapter) and its contents (UTXO, Chapter 7). What remains is the force that orders the backbone and decides contests: **proof-of-work expressed as weight**, the subject of Chapter 9.

1. A graph is a set of nodes connected by edges. It is an abstract structure for representing relationships; many systems (networks, maps, dependencies, ledgers) are naturally graphs. ↩
2. A node (in graph theory, also vertex) is one element of a graph. Hathor uses "vertex" for ledger nodes to avoid confusion with network nodes (the computers), which Chapter 6 defined separately. ↩
3. A directed graph has edges with a direction — each edge goes from one node to another, not symmetrically. An edge $A \rightarrow B$ does not imply $B \rightarrow A$. ↩
4. Acyclic means containing no cycles — no path that follows edges and returns to its starting node. A directed acyclic graph (DAG) combines directedness with acyclicity. ↩
5. A topological order is an arrangement of a DAG's nodes in a line such that every edge points from a later position to an earlier one — i.e. every node appears after all nodes it depends on. Only DAGs have one. It makes dependency-respecting processing possible. ↩
6. A vertex in `hathor-core` is any node of the ledger DAG — a block or a transaction. The umbrella term used when the kind does not matter; implemented as `GenericVertex` (aliased `BaseTransaction`). ↩

7. Parents are the earlier vertices a vertex attaches to and confirms — the DAG's structural (topology) edges. Stored as a list of vertex hashes. The generalization of a blockchain's single `prev` pointer. ↩
8. Tips are the vertices at the frontier of the DAG that no other vertex has yet confirmed (named as a parent). New vertices select their parents from among the tips. ↩
9. A vertex is confirmed by every later vertex that can reach it via parent edges. More confirmations (and accumulated work behind them) make a vertex harder to reverse — the DAG analogue of chain depth. ↩